

Microcontrollers and Embedded Systems Term Project

Password-Protected Irrigation Monitoring System (FRDM-KL25Z)

Mehmet Eralp Bulut | Student ID: 22303468

February 24, 2026

Intro Video: <https://www.youtube.com/watch?v=aw-JuuW3rH0>

Abstract

This project is an embedded irrigation monitoring prototype built on the FRDM-KL25Z. It combines a 16x2 LCD and 4x4 keypad UI with soil moisture measurement, light/rain detection, and a buzzer for alerts. The system is protected by a 4-digit password and shows clear status/alert screens for real-time monitoring.

1 Introduction

This portfolio project demonstrates a compact embedded system that monitors basic environmental conditions relevant to irrigation: soil moisture, ambient light, and rain. A keypad-based password locks the system until authorized, and a 16x2 LCD provides an easy-to-read interface. The push button switches between the main status screen and a rain status screen, while critical conditions trigger alert messages and an audible buzzer.

2 System Overview

Figure 1 summarizes the modules and signals.

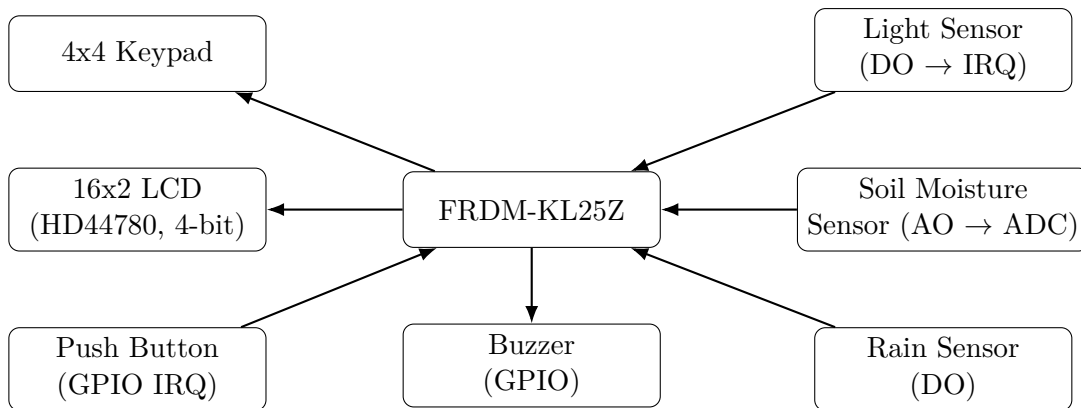


Figure 1: High-level block diagram of the irrigation monitoring system.

3 Hardware

3.1 Main Components

- FRDM-KL25Z microcontroller board
- 16x2 HD44780-compatible LCD (4-bit mode)
- 4x4 keypad (password entry)
- Soil moisture sensor (analog output to ADC)
- Light sensor (digital output with interrupt)
- Rain sensor (digital output)
- Push button (screen toggle) and buzzer (alert)

3.2 Connections (Summary)

- LCD via GPIO (4-bit data + control)
- Keypad via GPIO (row outputs, column inputs with pull-ups)
- Soil moisture to ADC input, light to GPIO interrupt, rain to GPIO input
- Buzzer to GPIO output, button to GPIO interrupt

4 Software Design

4.1 Timing Base (SysTick)

A 1ms timebase is created using SysTick interrupts. This timebase is used for:

- Non-blocking buzzer timeout (auto-off at 500ms).
- Periodic polling intervals (rain: 100ms, soil ADC: 250ms, UI refresh: 300ms).
- Debouncing of button and interrupt events.

4.2 Interrupt Usage

This project uses hardware interrupts to react quickly to user input and sensor events:

- **SysTick interrupt (1 ms timebase):** Implemented in `SysTick_Handler()`, used for timing, delays, debouncing, and buzzer timeout control.
- **PORTA interrupt (Light sensor on PTA12):** Configured in `Initialize_LightSensor_Interrupt()` and handled in `PORTA_IRQHandler()`. It triggers when the light sensor changes state and immediately switches the UI to the light alert mode and starts the buzzer.
- **PORTD interrupt (Push button on PTD4):** Configured in `Initialize_Button_Interrupt()` and handled in `PORTD_IRQHandler()`. It is used to toggle between the main screen and the rain status screen (only when the system is unlocked and not in an alert state).

4.3 User Interface and Modes

The LCD UI uses a `screen_mode` variable:

- **0: Normal screen** (moisture percent and light status)
- **1: Moisture alert** (Not Enough Moisture!)
- **2: Light alert** (Not Enough Light!)
- **4: Rain screen** (RAIN/DRY)

Alerts override the normal operation: if light is low, the system switches to light alert mode and blocks screen toggling until light is restored.

4.4 Password Protection

Before entering normal operation, the system requires a fixed 4-digit password (1234). Digits are displayed on the LCD as they are typed. On incorrect entry, the buzzer sounds and the user is prompted again.

4.5 Sensor Processing

Soil Moisture (ADC). The soil sensor is read via `ADC0_SE0`. The raw ADC is mapped into a percentage using two calibration points: `ADC_DRY` and `ADC_WET`. Hysteresis is implemented:

- Enter low-moisture alert at `MOIST_LOW_ENTER_PCT`.
- Exit low-moisture alert at `MOIST_LOW_EXIT_PCT`.

Light Sensor (Interrupt). The light sensor is configured on `PTA12` with interrupt-on-either-edge. On low-light detection, the system:

- Switches to mode 2 (Light alert).
- Activates buzzer for 500ms.

Rain Sensor (Digital Input). The rain sensor digital output is polled periodically. The firmware supports active-low modules via `RAIN_ACTIVE_LOW`.

5 Results

- Password gate works; wrong entry triggers buzzer and retry prompt.
- Normal screen shows moisture percentage and light status.
- Low moisture and low light override the UI with clear alert messages + buzzer.
- Push button toggles between main and rain status screens (when not in alerts).
- Rain status updates reliably during operation.

6 Conclusion

A practical irrigation monitoring prototype was implemented on the FRDM-KL25Z using an LCD/keypad interface, multiple sensors, and buzzer-based alerts. The project demonstrates embedded UI design, interrupt handling, ADC sensing, and reliable state-based behavior in a compact portfolio-ready system.

A Firmware Source Code

Listing 1: Firmware (KL25Z / MKL25Z4).

```
1 #include <MKL25Z4.h>
2 #include <stdint.h>
3
4 /* =====
5    SysTick: 1ms timebase
6    ===== */
7 volatile uint32_t system_ms_counter = 0;
8
9 void SysTick_Handler(void) { system_ms_counter++; }
10
11 static void delay_ms(uint32_t ms) {
12     uint32_t t0 = system_ms_counter;
13     while ((system_ms_counter - t0) < ms) { }
14 }
15
16 static void Initialize_SysTick(void) {
17     SystemCoreClockUpdate();
18     SysTick_Config(SystemCoreClock / 1000u);
19 }
20
21 /* =====
22    LCD (HD44780, 4-bit)
23    RS -> PTB0
24    E  -> PTB1
25    D4 -> PTB2
26    D5 -> PTB3
27    D6 -> PTC2
28    D7 -> PTC1
29    ===== */
30 #define PIN_LCD_RS  0u
31 #define PIN_LCD_E   1u
32 #define PIN_LCD_D4  2u
33 #define PIN_LCD_D5  3u
34 #define PIN_LCD_D6  2u    /* PORTC */
35 #define PIN_LCD_D7  1u    /* PORTC */
36
37 #define MASK_RS    (1u << PIN_LCD_RS)
38 #define MASK_E     (1u << PIN_LCD_E)
39 #define MASK_D4    (1u << PIN_LCD_D4)
40 #define MASK_D5    (1u << PIN_LCD_D5)
41 #define MASK_D6    (1u << PIN_LCD_D6)
42 #define MASK_D7    (1u << PIN_LCD_D7)
43
44 static void LCD_ToggleEnable(void);
45 static void LCD_TransmitNibble(uint8_t nibble);
46 static void LCD_TransmitByte(uint8_t byte_val, uint8_t rs);
47
48 static void Initialize_LCD(void);
49 static void LCD_SendCommand(uint8_t cmd);
50 static void LCD_SendData(uint8_t data);
51 static void LCD_WriteString(const char *s);
52 static void LCD_MoveCursor(uint8_t row, uint8_t col);
53 static void LCD_ClearAndWrite(uint8_t row, const char *text);
54
```

```

55 static void LCD_WriteNumber(uint32_t number) {
56     char buf[11];
57     int i = 0;
58     if (number == 0) { LCD_SendData('0'); return; }
59     while (number && i < 10) { buf[i++] = (char)('0' + (number % 10u));
        number /= 10u; }
60     for (int j = i - 1; j >= 0; j--) LCD_SendData((uint8_t)buf[j]);
61 }
62
63 static void Initialize_LCD(void) {
64     SIM->SCGC5 |= (SIM_SCGC5_PORTB_MASK | SIM_SCGC5_PORTC_MASK);
65
66     PORTB->PCR[PIN_LCD_RS] = PORT_PCR_MUX(1);
67     PORTB->PCR[PIN_LCD_E] = PORT_PCR_MUX(1);
68     PORTB->PCR[PIN_LCD_D4] = PORT_PCR_MUX(1);
69     PORTB->PCR[PIN_LCD_D5] = PORT_PCR_MUX(1);
70     PORTC->PCR[PIN_LCD_D6] = PORT_PCR_MUX(1);
71     PORTC->PCR[PIN_LCD_D7] = PORT_PCR_MUX(1);
72
73     PTB->PDDR |= (MASK_RS | MASK_E | MASK_D4 | MASK_D5);
74     PTC->PDDR |= (MASK_D6 | MASK_D7);
75
76     delay_ms(40);
77
78     LCD_TransmitNibble(0x3); delay_ms(5);
79     LCD_TransmitNibble(0x3); delay_ms(1);
80     LCD_TransmitNibble(0x3); delay_ms(1);
81     LCD_TransmitNibble(0x2);
82
83     LCD_SendCommand(0x28);
84     LCD_SendCommand(0x0C);
85     LCD_SendCommand(0x06);
86     LCD_SendCommand(0x01);
87     delay_ms(2);
88 }
89
90 static void LCD_ToggleEnable(void) {
91     PTB->PSOR = MASK_E;
92     for (volatile int d = 0; d < 50; d++) __NOP();
93     PTB->PCOR = MASK_E;
94     for (volatile int d = 0; d < 50; d++) __NOP();
95 }
96
97 static void LCD_TransmitNibble(uint8_t nibble) {
98     PTB->PCOR = (MASK_D4 | MASK_D5);
99     PTC->PCOR = (MASK_D6 | MASK_D7);
100
101     if (nibble & 0x01u) PTB->PSOR = MASK_D4;
102     if (nibble & 0x02u) PTB->PSOR = MASK_D5;
103     if (nibble & 0x04u) PTC->PSOR = MASK_D6;
104     if (nibble & 0x08u) PTC->PSOR = MASK_D7;
105
106     LCD_ToggleEnable();
107 }
108
109 static void LCD_TransmitByte(uint8_t byte_val, uint8_t rs) {
110     if (rs) PTB->PSOR = MASK_RS;
111     else PTB->PCOR = MASK_RS;

```

```

112     LCD_TransmitNibble(byte_val >> 4);
113     LCD_TransmitNibble(byte_val & 0x0Fu);
114 }
115
116
117 static void LCD_SendCommand(uint8_t cmd) {
118     LCD_TransmitByte(cmd, 0);
119     if (cmd == 0x01 || cmd == 0x02) delay_ms(2);
120     else delay_ms(1);
121 }
122
123 static void LCD_SendData(uint8_t data) {
124     LCD_TransmitByte(data, 1);
125     delay_ms(1);
126 }
127
128 static void LCD_WriteString(const char *s) {
129     while (*s) LCD_SendData((uint8_t)(*s++));
130 }
131
132 static void LCD_MoveCursor(uint8_t row, uint8_t col) {
133     uint8_t base = (row == 0) ? 0x00 : 0x40;
134     LCD_SendCommand((uint8_t)(0x80 | (base + col)));
135 }
136
137 static void LCD_ClearAndWrite(uint8_t row, const char *text) {
138     LCD_MoveCursor(row, 0);
139     int pos = 0;
140     while (text[pos] && pos < 16) LCD_SendData((uint8_t)text[pos++]);
141     while (pos < 16) { LCD_SendData(' '); pos++; }
142 }
143
144 /* =====
145    Button (PTD4) interrupt
146    Wire: PTD4 -> button -> GND
147    ===== */
148 #define BTN_PIN    4u
149 #define BTN_MASK    (1u << BTN_PIN)
150
151 /* =====
152    Digital Light Sensor interrupt on PORTA (PTA12)
153    Wire: OUT -> PTA12, VCC -> 3.3V, GND -> GND
154    ===== */
155 #define LIGHT_PIN_A    12u
156 #define LIGHT_MASK_A    (1u << LIGHT_PIN_A)
157 #define LIGHT_ACTIVE_LOW    1u
158
159 /* =====
160    Soil moisture ANALOG (AO) -> ADC on PTE20 (ADC0_SE0)
161    Wire: AO -> PTE20, VCC -> 3.3V, GND -> GND
162    ===== */
163 #define SOIL_ADC_PTE20_PIN    20u
164
165 #define ADC_DRY    3020u
166 #define ADC_WET    1000u
167 #define MOIST_INVERT    1u
168
169 #define MOIST_LOW_ENTER_PCT    35u

```

```

170 #define MOIST_LOW_EXIT_PCT    40u
171
172 /* =====
173  Rain sensor (DO) -> PTD6
174  Rain module pins: VCC, GND, DO, AO
175  Use DO only:
176      DO -> PTD6
177      VCC -> 3.3V
178      GND -> GND
179  Many rain modules are ACTIVE-LOW on DO (wet -> 0 suggests rain).
180  If yours is opposite, set RAIN_ACTIVE_LOW to 0.
181  ===== */
182 #define RAIN_PIN                6u
183 #define RAIN_MASK                (1u << RAIN_PIN)
184 #define RAIN_ACTIVE_LOW        1u
185
186 volatile uint8_t rain_is_detected = 0;
187
188 static inline uint8_t Read_RainRaw(void) {
189     return (PTD->PDIR & RAIN_MASK) ? 1u : 0u;
190 }
191
192 static void Update_RainFlag_FromPin(void) {
193     uint8_t raw = Read_RainRaw();
194     #if RAIN_ACTIVE_LOW
195         rain_is_detected = (raw == 0u) ? 1u : 0u;
196     #else
197         rain_is_detected = (raw == 1u) ? 1u : 0u;
198     #endif
199 }
200
201 static void Initialize_Rain_DO_Input(void) {
202     SIM->SCGC5 |= SIM_SCGC5_PORTD_MASK;
203     PORTD->PCR[RAIN_PIN] = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK |
        PORT_PCR_PS_MASK;
204     PTD->PDDR &= ~RAIN_MASK;
205     Update_RainFlag_FromPin();
206 }
207
208 /* =====
209  Buzzer (PTD5)
210  ===== */
211 #define BUZZER_PIN            5u
212 #define BUZZER_MASK            (1u << BUZZER_PIN)
213
214 static void Initialize_Buzzer(void) {
215     SIM->SCGC5 |= SIM_SCGC5_PORTD_MASK;
216     PORTD->PCR[BUZZER_PIN] = PORT_PCR_MUX(1);
217     PTD->PDDR |= BUZZER_MASK;
218     PTD->PCOR = BUZZER_MASK;
219 }
220
221 static inline void Buzzer_On(void) { PTD->PSOR = BUZZER_MASK; }
222 static inline void Buzzer_Off(void) { PTD->PCOR = BUZZER_MASK; }
223
224 volatile uint8_t buzzer_active = 0;
225 volatile uint32_t buzzer_start_time = 0;
226

```

```

227 static void Start_Buzzer_500ms(void) {
228     Buzzer_On();
229     buzzer_active = 1u;
230     buzzer_start_time = system_ms_counter;
231 }
232
233 /* Service buzzer timeout even during blocking password flow */
234 static void Buzzer_Service(void) {
235     if (buzzer_active && (system_ms_counter - buzzer_start_time) >= 500
236         u) {
237         Buzzer_Off();
238         buzzer_active = 0u;
239     }
240 }
241
242 /* Delay while still allowing buzzer to auto-stop */
243 static void delay_ms_service_buzzer(uint32_t ms) {
244     uint32_t t0 = system_ms_counter;
245     while ((system_ms_counter - t0) < ms) {
246         Buzzer_Service();
247     }
248 }
249
250 /* =====
251 4x4 Keypad (Password)
252 ROWS outputs: PTA1, PTA2, PTA4, PTA5
253 COLS inputs+pullups: PTC3, PTC4, PTC5, PTC6
254 ===== */
255 #define KP_ROW0_PA1 1u
256 #define KP_ROW1_PA2 2u
257 #define KP_ROW2_PA4 4u
258 #define KP_ROW3_PA5 5u
259
260 #define KP_COL0_PC3 3u
261 #define KP_COL1_PC4 4u
262 #define KP_COL2_PC5 5u
263 #define KP_COL3_PC6 6u
264
265 #define KP_ROW0_MASK (1u << KP_ROW0_PA1)
266 #define KP_ROW1_MASK (1u << KP_ROW1_PA2)
267 #define KP_ROW2_MASK (1u << KP_ROW2_PA4)
268 #define KP_ROW3_MASK (1u << KP_ROW3_PA5)
269
270 #define KP_COL0_MASK (1u << KP_COL0_PC3)
271 #define KP_COL1_MASK (1u << KP_COL1_PC4)
272 #define KP_COL2_MASK (1u << KP_COL2_PC5)
273 #define KP_COL3_MASK (1u << KP_COL3_PC6)
274
275 static const char keypad_map[4][4] = {
276     {'1','2','3','A'},
277     {'4','5','6','B'},
278     {'7','8','9','C'},
279     {'*','0','#','D'}
280 };
281
282 static void Keypad_Init(void) {
283     SIM->SCGC5 |= (SIM_SCGC5_PORTA_MASK | SIM_SCGC5_PORTC_MASK);

```



```

284 PORTA->PCR[KP_ROW0_PA1] = PORT_PCR_MUX(1);
285 PORTA->PCR[KP_ROW1_PA2] = PORT_PCR_MUX(1);
286 PORTA->PCR[KP_ROW2_PA4] = PORT_PCR_MUX(1);
287 PORTA->PCR[KP_ROW3_PA5] = PORT_PCR_MUX(1);
288
289 PTA->PDDR |= (KP_ROW0_MASK | KP_ROW1_MASK | KP_ROW2_MASK |
                KP_ROW3_MASK);
290 PTA->PSOR = (KP_ROW0_MASK | KP_ROW1_MASK | KP_ROW2_MASK |
                KP_ROW3_MASK);
291
292 PORTC->PCR[KP_COLO_PC3] = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK |
                PORT_PCR_PS_MASK;
293 PORTC->PCR[KP_COL1_PC4] = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK |
                PORT_PCR_PS_MASK;
294 PORTC->PCR[KP_COL2_PC5] = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK |
                PORT_PCR_PS_MASK;
295 PORTC->PCR[KP_COL3_PC6] = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK |
                PORT_PCR_PS_MASK;
296
297 PTC->PDDR &= ~(KP_COLO_MASK | KP_COL1_MASK | KP_COL2_MASK |
                KP_COL3_MASK);
298 }
299
300 static inline void Keypad_AllRowsHigh(void) {
301     PTA->PSOR = (KP_ROW0_MASK | KP_ROW1_MASK | KP_ROW2_MASK |
                KP_ROW3_MASK);
302 }
303
304 static inline void Keypad_SelectRow(uint8_t r) {
305     Keypad_AllRowsHigh();
306     if (r == 0u) PTA->PCOR = KP_ROW0_MASK;
307     else if (r == 1u) PTA->PCOR = KP_ROW1_MASK;
308     else if (r == 2u) PTA->PCOR = KP_ROW2_MASK;
309     else
310         PTA->PCOR = KP_ROW3_MASK;
311 }
312
313 static inline uint8_t Keypad_ReadCols(void) {
314     uint32_t v = PTC->PDIR;
315     uint8_t c0 = (v & KP_COLO_MASK) ? 1u : 0u;
316     uint8_t c1 = (v & KP_COL1_MASK) ? 1u : 0u;
317     uint8_t c2 = (v & KP_COL2_MASK) ? 1u : 0u;
318     uint8_t c3 = (v & KP_COL3_MASK) ? 1u : 0u;
319     return (uint8_t)(((!c0) << 0) | ((!c1) << 1) | ((!c2) << 2) | ((!c3)
        << 3));
320 }
321
322 static char Keypad_GetKey_Blocking(void) {
323     while (1) {
324         for (uint8_t r = 0; r < 4u; r++) {
325             Keypad_SelectRow(r);
326             delay_ms(2);
327
328             uint8_t cols = Keypad_ReadCols();
329             if (cols != 0u) {
330                 delay_ms(20);
331                 cols = Keypad_ReadCols();
332                 if (cols == 0u) continue;

```

```

333         uint8_t c = 0u;
334         if      (cols & 0x01u) c = 0u;
335         else if (cols & 0x02u) c = 1u;
336         else if (cols & 0x04u) c = 2u;
337         else      c = 3u;
338
339         char key = keypad_map[r][c];
340
341         while (Keypad_ReadCols() != 0u) { delay_ms(5); }
342         Keypad_AllRowsHigh();
343         return key;
344     }
345 }
346 }
347 }
348
349 /* =====
350    Fixed password = 1234, show typed digits
351    ===== */
352 static const char FIXED_PASS[4] = {'1','2','3','4'};
353 static char pass_try[4];
354 static volatile uint8_t unlocked = 0;
355
356 /* =====
357    Global state / modes
358    0 = Normal screen (main)
359    1 = Moisture alert
360    2 = No light alert (sensor-only)
361    3 = Movement alert (unused now)
362    4 = Rain screen (main)
363    ===== */
364 volatile uint8_t screen_mode = 0;
365 volatile uint8_t light_is_low = 0;
366
367 static inline uint8_t Read_LightRaw(void) {
368     return (PTA->PDIR & LIGHT_MASK_A) ? 1u : 0u;
369 }
370
371 static void Update_LightFlag_FromPin(void) {
372     uint8_t raw = Read_LightRaw();
373     #if LIGHT_ACTIVE_LOW
374         light_is_low = (raw == 0u) ? 1u : 0u;
375     #else
376         light_is_low = (raw == 1u) ? 1u : 0u;
377     #endif
378 }
379
380 /* ----- ADC for soil moisture ----- */
381 static void Initialize_ADC_Soil(void) {
382     SIM->SCGC5 |= SIM_SCGC5_PORTE_MASK;
383     SIM->SCGC6 |= SIM_SCGC6_ADC0_MASK;
384
385     PORTE->PCR[SOIL_ADC_PTE20_PIN] = 0;
386
387     ADC0->CFG1 = ADC_CFG1_ADIV(1) | ADC_CFG1_MODE(1) | ADC_CFG1_ADICLK
388         (0);
389     ADC0->CFG2 = ADC_CFG2_ADLSTS(3);

```

```

390     ADC0->SC2 = 0;
391     ADC0->SC3 = ADC_SC3_AVGE_MASK | ADC_SC3_AVGS(3);
392 }
393
394 static uint16_t Read_Soil_ADC12(void) {
395     ADC0->SC1[0] = ADC_SC1_ADCH(0);
396     while (!(ADC0->SC1[0] & ADC_SC1_COCON_MASK)) { }
397     return (uint16_t)ADC0->R[0];
398 }
399
400 static uint32_t Soil_MoisturePercent(uint16_t adc) {
401     uint32_t dry = ADC_DRY;
402     uint32_t wet = ADC_WET;
403     if (dry == wet) return 0u;
404
405     #if MOIST_INVERT
406         if ((uint32_t)adc >= dry) return 0u;
407         if ((uint32_t)adc <= wet) return 100u;
408         return (uint32_t)(((dry - (uint32_t)adc) * 100u) / (dry - wet));
409     #else
410         if ((uint32_t)adc <= dry) return 0u;
411         if ((uint32_t)adc >= wet) return 100u;
412         return (uint32_t)((((uint32_t)adc - dry) * 100u) / (wet - dry));
413     #endif
414 }
415
416 /* =====
417 Screens
418 ===== */
419 static void Show_NormalScreen(uint32_t moist_pct) {
420     LCD_ClearAndWrite(0, "Irrigation Sys");
421
422     LCD_MoveCursor(1, 0);
423     LCD_SendData(' ');
424     LCD_WriteString("M=");
425     LCD_WriteNumber(moist_pct);
426     LCD_WriteString("% L=");
427     LCD_WriteString(light_is_low ? "LOW" : "OK ");
428 }
429
430 static void Show_RainScreen(void) {
431     LCD_ClearAndWrite(0, "Rain Sensor");
432
433     LCD_MoveCursor(1, 0);
434     LCD_SendData(' ');
435     LCD_WriteString("Status: ");
436     LCD_WriteString(rain_is_detected ? "RAIN" : "DRY ");
437 }
438
439 static void Show_MoistureAlert(void) {
440     LCD_ClearAndWrite(0, "Not Enough");
441     LCD_ClearAndWrite(1, "Moisture!");
442 }
443
444 static void Show_LightAlert(void) {
445     LCD_ClearAndWrite(0, "Not Enough");
446     LCD_ClearAndWrite(1, "Light!");
447 }

```

```

448
449 static void Show_MovementAlert(void) {
450     LCD_ClearAndWrite(0, "Movement");
451     LCD_ClearAndWrite(1, "Detected!");
452 }
453
454 /* Password UI: show typed digits */
455 static void Show_PasswordPrompt(uint8_t entered) {
456     LCD_ClearAndWrite(0, "Enter Password");
457     LCD_MoveCursor(1, 0);
458     LCD_WriteString("PASS: ");
459     for (uint8_t i = 0; i < 4u; i++) {
460         if (i < entered) LCD_SendData((uint8_t)pass_try[i]);
461         else LCD_SendData('_');
462     }
463     LCD_WriteString("    ");
464 }
465
466 static void Password_RunFlow(void) {
467     while (!unlocked) {
468         uint8_t idx = 0;
469         Show_PasswordPrompt(idx);
470
471         while (idx < 4u) {
472             char k = Keypad_GetKey_Blocking();
473             if (k >= '0' && k <= '9') {
474                 pass_try[idx++] = k;
475                 Show_PasswordPrompt(idx);
476             }
477         }
478
479         uint8_t ok = 1u;
480         for (uint8_t i = 0; i < 4u; i++) {
481             if (pass_try[i] != FIXED_PASS[i]) { ok = 0u; break; }
482         }
483
484         if (ok) {
485             LCD_ClearAndWrite(0, "Password OK");
486             LCD_ClearAndWrite(1, "System Ready");
487             unlocked = 1u;
488             delay_ms_service_buzzer(800);
489         } else {
490             LCD_ClearAndWrite(0, "Wrong Password");
491             LCD_ClearAndWrite(1, "Try Again");
492             Start_Buzzer_500ms();
493             delay_ms_service_buzzer(1000);
494         }
495     }
496 }
497
498 /* =====
499 Init: Button IRQ (PORTD)
500 ===== */
501 static void Initialize_Button_Interrupt(void) {
502     SIM->SCGC5 |= SIM_SCGC5_PORTD_MASK;
503
504     PORTD->PCR[BTN_PIN] =
505     PORT_PCR_MUX(1) |

```

```

506     PORT_PCR_PE_MASK |
507     PORT_PCR_PS_MASK |
508     PORT_PCR_IRQC(0xAu);
509
510     PTD->PDDR &= ~BTN_MASK;
511
512     PORTD->ISFR = BTN_MASK;
513     NVIC_EnableIRQ(PORTD_IRQn);
514 }
515
516 /* =====
517  Init: Light IRQ (PORTA)
518  ===== */
519 static void Initialize_LightSensor_Interrupt(void) {
520     SIM->SCGC5 |= SIM_SCGC5_PORTA_MASK;
521
522     PORTA->PCR[LIGHT_PIN_A] =
523         PORT_PCR_MUX(1) |
524         PORT_PCR_PE_MASK |
525         PORT_PCR_PS_MASK |
526         PORT_PCR_IRQC(0xBu);
527
528     PTA->PDDR &= ~LIGHT_MASK_A;
529
530     Update_LightFlag_FromPin();
531
532     PORTA->ISFR = LIGHT_MASK_A;
533     NVIC_EnableIRQ(PORTA_IRQn);
534 }
535
536 /* =====
537  ISRs
538  ===== */
539 void PORTD_IRQHandler(void) {
540     if (PORTD->ISFR & BTN_MASK) {
541         PORTD->ISFR = BTN_MASK;
542
543         if (!unlocked) return;
544         if (screen_mode == 1u || screen_mode == 2u) return;
545         if (light_is_low) return;
546
547         static uint32_t last_ms = 0;
548         uint32_t now = system_ms_counter;
549         if ((now - last_ms) < 200u) return;
550         last_ms = now;
551
552         if (screen_mode == 0u) screen_mode = 4u;
553         else screen_mode = 0u;
554     }
555 }
556
557 void PORTA_IRQHandler(void) {
558     if (PORTA->ISFR & LIGHT_MASK_A) {
559         PORTA->ISFR = LIGHT_MASK_A;
560
561         static uint32_t last_ms = 0;
562         uint32_t now = system_ms_counter;
563         if ((now - last_ms) < 50u) return;

```

```

564         last_ms = now;
565
566         Update_LightFlag_FromPin();
567
568         if (!unlocked) return;
569
570         if (light_is_low) {
571             screen_mode = 2u;
572             Start_Buzzer_500ms();
573         } else {
574             if (screen_mode == 2u) screen_mode = 0u;
575         }
576     }
577 }
578
579 /* =====
580    main
581    ===== */
582 int main(void) {
583     SystemCoreClockUpdate();
584
585     Initialize_SysTick();
586     Initialize_LCD();
587     Initialize_Buzzer();
588     Keypad_Init();
589
590     Initialize_LightSensor_Interrupt();
591     Initialize_Button_Interrupt();
592     Initialize_ADC_Soil();
593     Initialize_Rain_D0_Input();
594
595     LCD_ClearAndWrite(0, "Irrigation Sys");
596     LCD_ClearAndWrite(1, "Locked");
597     delay_ms_service_buzzer(600);
598
599     Password_RunFlow();
600
601     LCD_ClearAndWrite(0, "Irrigation Sys");
602     LCD_ClearAndWrite(1, "Unlocked!");
603     delay_ms_service_buzzer(600);
604
605     uint8_t last_mode = 255u;
606     uint32_t moist_pct = 0;
607     uint8_t moisture_low = 0;
608
609     while (1) {
610         /* buzzer auto-off after 500ms (works everywhere) */
611         Buzzer_Service();
612
613         /* Poll rain DO */
614         static uint32_t last_rain_ms = 0;
615         if ((system_ms_counter - last_rain_ms) >= 100u) {
616             last_rain_ms = system_ms_counter;
617             Update_RainFlag_FromPin();
618         }
619
620         /* Update soil moisture */
621         static uint32_t last_soil_ms = 0;

```

```

622     if ((system_ms_counter - last_soil_ms) >= 250u) {
623         last_soil_ms = system_ms_counter;
624
625         uint16_t adc = Read_Soil_ADC12();
626         moist_pct = Soil_MoisturePercent(adc);
627
628         if (!moisture_low && moist_pct <= MOIST_LOW_ENTER_PCT) {
629             moisture_low = 1u;
630             if (!light_is_low) {
631                 screen_mode = 1u;
632                 Start_Buzzer_500ms();
633             }
634         } else if (moisture_low && moist_pct >= MOIST_LOW_EXIT_PCT)
635         {
636             moisture_low = 0u;
637             if (!light_is_low && screen_mode == 1u) {
638                 screen_mode = 0u;
639             }
640         }
641
642         /* refresh main screens */
643         static uint32_t last_refresh = 0;
644         if ((system_ms_counter - last_refresh) >= 300u) {
645             last_refresh = system_ms_counter;
646
647             if (screen_mode == 0u) Show_NormalScreen(moist_pct);
648             else if (screen_mode == 4u) Show_RainScreen();
649         }
650
651         /* redraw on mode change */
652         if (screen_mode != last_mode) {
653             last_mode = screen_mode;
654
655             if (screen_mode == 0u) Show_NormalScreen(moist_pct);
656             else if (screen_mode == 4u) Show_RainScreen();
657             else if (screen_mode == 1u) Show_MoistureAlert();
658             else if (screen_mode == 2u) Show_LightAlert();
659             else
660                 Show_MovementAlert();
661         }
662         delay_ms(20);
663     }
664 }

```